JavaScript is disabled on your browser.

compact1, compact2, compact3

java.io

Class SequenceInputStream

java.lang.Object
    java.io.InputStream
        java.io.SequenceInputStream

All Implemented Interfaces:

`Closeable, AutoCloseable`

```
public class SequenceInputStream
extends InputStream
```

A `SequenceInputStream` represents the logical concatenation of other input streams. It starts out with an ordered collection of input streams and reads from the first one until end of file is reached, whereupon it reads from the second one, and so on, until end of file is reached on the last of the contained input streams.

Since:

`JDK1.0`

Constructor Summary

Constructors

Constructor	Description
`SequenceInputStream(Enumeration<? extends InputStream> e)`	Initializes a newly created `SequenceInputStream` by remembering the argument, which must be an `Enumeration` that produces objects

	whose run-time type is `InputStream`.
`SequenceInputStream(InputStream s1, InputStream s2)`	Initializes a newly created `SequenceInputStream` by remembering the two arguments, which will be read in order, first `s1` and then `s2`, to provide the bytes to be read from this `SequenceInputStream`.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`int`	`available()`	Returns an estimate of the number of bytes that can be read (or skipped over) from the current underlying input stream without blocking by the next invocation of a method for the current underlying input stream.
`void`	`close()`	Closes this input stream and releases any system resources associated with the stream.
`int`	`read()`	Reads the next byte of data from this input stream.
`int`	`read(byte[] b, int off, int len)`	Reads up to `len` bytes of data from this input stream into an array of bytes.

Methods inherited from class java.io.InputStream

`mark`, `markSupported`, `read`, `reset`, `skip`

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

SequenceInputStream

```
public SequenceInputStream(Enumeration<? extends InputStream> e)
```

Initializes a newly created `SequenceInputStream` by remembering the argument, which must be an `Enumeration` that produces objects whose run-time type is `InputStream`. The input streams that are produced by the enumeration will be read, in order, to provide the bytes to be read from this `SequenceInputStream`. After each input stream from the enumeration is exhausted, it is closed by calling its `close` method.

Parameters:

`e - an enumeration of input streams.`

See Also:

`Enumeration`

SequenceInputStream

```
public SequenceInputStream(InputStream s1,
                           InputStream s2)
```

Initializes a newly created `SequenceInputStream` by remembering the two arguments, which will be read in order, first `s1` and then `s2`, to provide the bytes to be read from this `SequenceInputStream`.

Parameters:

`s1 - the first input stream to read.`

`s2 - the second input stream to read.`

Method Detail

available

```
public int available()
             throws IOException
```

Returns an estimate of the number of bytes that can be read (or skipped over) from the current underlying input stream without blocking by the next invocation of a method for the current underlying input stream. The next invocation might be the same thread or another thread. A single read or skip of this many bytes will not block, but may read or skip fewer bytes.

This method simply calls `available` of the current underlying input stream and returns the result.

Overrides:

`available` in class `InputStream`

Returns:

an estimate of the number of bytes that can be read (or skipped over)
from the current underlying input stream without blocking or 0 if this input stream has been closed by invoking its `close()` method

Throws:

`IOException` - if an I/O error occurs.

Since:

JDK1.1

read

```
public int read()
         throws IOException
```

Reads the next byte of data from this input stream. The byte is returned as an `int` in the range `0` to `255`. If no byte is available because the end of the stream has been reached, the value `-1` is returned. This method blocks until input data is available, the end of the stream is detected, or an exception is thrown.

This method tries to read one character from the current substream. If it reaches the end of the stream, it calls the `close` method of the current substream and begins reading from the next substream.

Specified by:

`read` in class `InputStream`

Returns:

the next byte of data, or -1 if the end of the stream is reached.

Throws:

IOException - if an I/O error occurs.

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Reads up to `len` bytes of data from this input stream into an array of bytes. If `len` is not zero, the method blocks until at least 1 byte of input is available; otherwise, no bytes are read and `0` is returned.

The `read` method of `SequenceInputStream` tries to read the data from the current substream. If it fails to read any characters because the substream has reached the end of the stream, it calls the `close` method of the current substream and begins reading from the next substream.

Overrides:

read in class InputStream

Parameters:

b - the buffer into which the data is read.

off - the start offset in array b at which the data is written.

len - the maximum number of bytes read.

Returns:

int the number of bytes read.

Throws:

NullPointerException - If b is null.

IndexOutOfBoundsException - If off is negative, len is negative, or len is greater than b.length - off

IOException - if an I/O error occurs.

See Also:

InputStream.read()

close

```
public void close()
           throws IOException
```

Closes this input stream and releases any system resources associated with the stream. A closed `SequenceInputStream` cannot perform input operations and cannot be reopened.

If this stream was created from an enumeration, all remaining elements are requested from the enumeration and closed before the `close` method returns.

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `AutoCloseable`

Overrides:

`close` in class `InputStream`

Throws:

`IOException` - if an I/O error occurs.

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.